

Aula 08 — Métricas para Classificação

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Leitura recomendada: AME §7.4 e cap. 9; ISLP §4.4–4.5

Objetivos da aula

Ao final desta aula o aluno deve ser capaz de:

- explicar por que a *acurácia* é insuficiente, sobretudo em dados desbalanceados;
- ler uma **matriz de confusão** e calcular sensibilidade, especificidade, precisão, VPN e F_1 ;
- formular **perdas assimétricas** e derivar o *corte ótimo* de um classificador *plug-in*;
- construir e interpretar a **curva ROC** e a **AUC**;
- escolher estratégias para **dados desbalanceados** (corte, pesos, re-amostragem) e entender que o problema real são os *custos*;
- distinguir *classificar bem* de *estimar bem probabilidades* (regras de pontuação próprias).

Em sala

Continuação direta da Aula 07: lá vimos que o classificador de Bayes minimiza a probabilidade de erro — mas a probabilidade de erro nem sempre é o que importa. Pergunta de abertura: “*um teste de câncer que diz ‘saúdável’ para todo mundo acerta 99% das vezes. Ele é bom?*”

1 Por que a acurácia não basta

O risco 0–1 (Aula 07) trata *todo* erro como igual. Mas:

- **Custos assimétricos:** num *screening*, deixar de detectar um doente (falso negativo) é muito pior que um alarme falso (falso positivo).
- **Dados desbalanceados:** com 10 doentes em 1000, o classificador trivial $g \equiv 0$ tem acurácia 99% — e sensibilidade zero. Como métodos buscam aproximar o classificador de Bayes, eles *tendem* a esse classificador inútil quando a classe é rara.

Atenção

A raiz do problema **não é** o desbalanceamento em si, mas o fato de que os *custos* dos erros são diferentes — de modo que o classificador de Bayes (que minimiza só a probabilidade de erro) deixa de ser o classificador desejado. Tenha isso em mente: as “correções” abaixo são formas de codificar custos.

2 Matriz de confusão e métricas

Para classificação binária ($Y \in \{0, 1\}$, “1” = positivo), cruzamos predito $\times$ verdadeiro:

		Verdadeiro	
		$Y = 0$	$Y = 1$
Predito	$\hat{g} = 0$	VN (verd. neg.)	FN (falso neg.)
	$\hat{g} = 1$	FP (falso pos.)	VP (verd. pos.)

A partir dela definem-se (cada uma estima uma probabilidade condicional populacional):

$$\text{Sensibilidade (recall): } S = \frac{VP}{VP + FN} \approx \mathbb{P}(\hat{g} = 1 \mid Y = 1),$$

$$\text{Especificidade: } E = \frac{VN}{VN + FP} \approx \mathbb{P}(\hat{g} = 0 \mid Y = 0),$$

$$\text{Precisão (VPP): } P = \frac{VP}{VP + FP} \approx \mathbb{P}(Y = 1 \mid \hat{g} = 1),$$

$$\text{Valor preditivo negativo: } VP_N = \frac{VN}{VN + FN} \approx \mathbb{P}(Y = 0 \mid \hat{g} = 0).$$

A estatística F_1 combina sensibilidade e precisão pela média harmônica:

$$F_1 = \frac{2}{1/S + 1/P} = \frac{2PS}{P + S}. \quad (1)$$

Ideia central

Sensibilidade olha “dos positivos reais, quantos peguei?”; *precisão* olha “dos que chamei de positivo, quantos acertei?”. O classificador trivial $g \equiv 0$ tem $S = 0$ e $E = 1$: a especificidade alta esconde que ele falha em *todos* os doentes. Por isso olhe *várias* métricas, não uma só.

Atenção

Todas essas quantidades devem ser calculadas no conjunto de *validação/teste*, nunca no treino (senão superestimam o desempenho — Aula 03).

3 Perdas assimétricas e o corte ótimo

Para codificar custos diferentes, redefina o risco com pesos $l_0, l_1 > 0$:

$$R(g) = l_1 \mathbb{P}(Y \neq g(\mathbf{X}), Y = 0) + l_0 \mathbb{P}(Y \neq g(\mathbf{X}), Y = 1), \quad (2)$$

onde l_0 penaliza errar um positivo (FN) e l_1 penaliza errar um negativo (FP).

Proposição 3.1 (Corte ótimo sob perda assimétrica). *O classificador que minimiza (2) é*

$$g(\mathbf{x}) = \mathbf{1} \left\{ \mathbb{P}(Y = 1 \mid \mathbf{x}) > \frac{l_1}{l_0 + l_1} \right\}.$$

Demonstração. Condicionando em $\mathbf{x}$, o custo esperado de prever 1 é $l_1 \mathbb{P}(Y = 0 \mid \mathbf{x})$ e o de prever 0 é $l_0 \mathbb{P}(Y = 1 \mid \mathbf{x})$. Prevemos 1 quando $l_1 \mathbb{P}(Y = 0 \mid \mathbf{x}) < l_0 \mathbb{P}(Y = 1 \mid \mathbf{x})$, isto é, $l_1(1 - \eta) < l_0 \eta$ com $\eta = \mathbb{P}(Y = 1 \mid \mathbf{x})$, o que dá $\eta > l_1/(l_0 + l_1)$. Como cada $\mathbf{x}$ é otimizado pontualmente, segue o resultado. $\square$

Ideia central

Eis a mensagem operacional: **mudar custos = mudar o corte**. Com $l_0 = l_1$ recuperamos o corte $1/2$ (Bayes). Se errar um positivo é mais caro ($l_0 > l_1$), o corte cai abaixo de $1/2$ — classificamos como positivo com menos evidência. Um caso notável: tomar $l_c = \pi_c$ leva ao corte $\hat{\mathbb{P}}(Y = 1)$ (a prevalência), em vez de $1/2$.

4 Escolhendo o corte: curva ROC e AUC

Um classificador *plug-in* probabilístico gera uma *família* de classificadores variando o corte K : $g_K(\mathbf{x}) = \mathbf{1}\{\hat{\mathbb{P}}(Y = 1 | \mathbf{x}) \geq K\}$. Cada K dá um par (sensibilidade, especificidade).

A **curva ROC** (*Receiver Operating Characteristic*) traça a sensibilidade ($\mathbb{P}(\hat{g} = 1 | Y = 1)$) contra 1–especificidade (a taxa de falso positivo) à medida que K varia de 1 a 0. Propriedades:

- o canto superior esquerdo $(0, 1)$ é o classificador perfeito;
- a diagonal é o “palpite aleatório”;
- a **AUC** (área sob a curva) resume o desempenho em um número: AUC = 1 é perfeito, 0,5 é aleatório. Interpretação: probabilidade de o modelo dar *score* maior a um positivo sorteado do que a um negativo sorteado.

Ideia central

A ROC/AUC avalia o *ordenamento* dado pelo modelo **independentemente do corte** — útil para comparar classificadores antes de fixar custos. Já escolhido o contexto, fixa-se K : critério comum é maximizar Sensibilidade + Especificidade (ponto mais próximo do canto $(0, 1)$), ou usar o corte da Prop. 3.1 se os custos são conhecidos.

Atenção

A curva ROC também deve ser construída na *validação/teste*. E cuidado: em dados muito desbalanceados, a ROC pode parecer otimista — a *curva precisão–recall* costuma ser mais informativa nesses casos.

5 Lidando com dados desbalanceados

Três famílias de estratégias (todas, no fundo, codificam custos):

Ajustar o corte

para métodos probabilísticos, basta mover K (Prop. 3.1). Em geral é a solução mais limpa.

Reponderar observações

dar peso maior à classe rara no treino, de modo que sua contribuição na função objetivo aumente (`class_weight` no `scikit-learn`). Útil quando o método não estima $\mathbb{P}(Y = 1 | \mathbf{x})$.

Re-amostrar

alterar artificialmente a prevalência: *oversampling* da minoria (ex.: SMOTE) ou *undersampling* da maioria.

Atenção

Repetindo a mensagem central: o “problema” não é o desbalanceamento, são os *custos desiguais*. Se os custos forem realmente iguais, um modelo bem calibrado com corte 1/2 pode ser o correto, mesmo desbalanceado. Não aplique re-amostragem reflexivamente.

6 Classificar bem $\neq$ estimar bem probabilidades

Um classificador *plug-in* pode estar muito perto do classificador de Bayes **sem** que $\hat{\mathbb{P}} \approx \mathbb{P}$: para a decisão, basta que $\mathbb{P}(Y = 1 | \mathbf{x}) - K$ tenha o *mesmo sinal* que $\mathbb{P}(Y = 1 | \mathbf{x}) - K$. Ou seja, boa classificação não garante boa estimativa de probabilidade.

Quando precisamos das *probabilidades* em si (para ordenar riscos, decidir com custos variáveis, ou inferência conformal), avaliamos com **regras de pontuação próprias**, que são minimizadas pela probabilidade verdadeira:

- **Entropia cruzada / log-loss**: $-\frac{1}{n} \sum_i [Y_i \log \hat{p}_i + (1 - Y_i) \log(1 - \hat{p}_i)]$ (é a log-verossimilhança negativa da logística);
- **Score de Brier**: $\frac{1}{n} \sum_i (\hat{p}_i - Y_i)^2$ (erro quadrático das probabilidades).

Esses critérios são os indicados para escolher *tuning parameters* de classificadores probabilísticos (ex.: λ da logística penalizada) quando o objetivo é estimar bem $\mathbb{P}$.

Ideia central

Calibração: um modelo é bem calibrado se, dentre os casos a que ele atribui $\hat{p} = 0,7$, cerca de 70% são de fato positivos. *Log-loss* e Brier penalizam a má calibração; acurácia e AUC, não. Escolha a métrica pelo objetivo real do problema.

7 Prática em Python

```

1 from sklearn.metrics import (confusion_matrix, classification_report,
2                               roc_auc_score, RocCurveDisplay,
3                               log_loss, brier_score_loss)
4
5 p = modelo.predict_proba(X_te)[: , 1] # probabilidades estimadas
6 y_hat = (p >= 0.5).astype(int) # corte; ajuste conforme os custos
7
8 print(confusion_matrix(y_te, y_hat))
9 print(classification_report(y_te, y_hat)) # precisao, recall, F1 por classe
10 print("AUC:", roc_auc_score(y_te, p))
11 print("log-loss:", log_loss(y_te, p),
12       " Brier:", brier_score_loss(y_te, p))
13 RocCurveDisplay.from_predictions(y_te, p)
14
15 # desbalanceamento: reponderar a classe rara
16 # LogisticRegression(class_weight="balanced")

```

O notebook Aula prática 08 e o material MatConf.pdf exploram a matriz de confusão e a ROC no bank_train_redux.csv.

Resumo

- Acurácia engana sob custos assimétricos/desbalanceamento; olhe a **matriz de confusão**.

- Sensibilidade, especificidade, precisão, VPN e F_1 (1) contam histórias diferentes.
- **Perda assimétrica** (2) $\Rightarrow$ corte ótimo $l_1/(l_0 + l_1)$ (Prop. 3.1): mudar custos é mudar o corte.
- **ROC/AUC** avaliam o ordenamento independentemente do corte; fixe K por custos ou por sensibilidade+especificidade.
- Desbalanceamento: corte, pesos ou re-amostragem — mas a raiz são os custos.
- Classificar bem $\neq$ estimar $\mathbb{P}$ bem: use *log-loss*/Brier e pense em **calibração**.

Leitura recomendada. [AME] §7.4 (matriz de confusão e métricas), §9.1 (perdas assimétricas, cortes, dados desbalanceados), §9.2 (classificação vs. estimação de probabilidades); [ISLP] §4.4–4.5 (matriz de confusão, ROC).